

```
@page { margin: 14mm 12mm; } body { font-family: "DejaVu Sans",
Arial, sans-serif; font-size: 10.5pt; line-height: 1.34; color: #1a2733;
max-width: 100%; margin: 0; } h1 { font-size: 19pt; color: #12324b;
border-bottom: 2px solid #12324b; padding-bottom: 3px; } h2 { font-
size: 14pt; color: #12324b; border-left: 4px solid #12324b; padding-
left: 7px; margin-top: 16px; } h3 { font-size: 12pt; color: #234;
margin-top: 12px; } table { border-collapse: collapse; width: 100%;
margin: 8px 0; font-size: 9pt; } th, td { border: 1px solid #b7c2cc;
padding: 3px 6px; text-align: left; vertical-align: top; } th {
background: #12324b; color: #fff; } tr:nth-child(even) td {
background: #f2f5f7; } code { background: #eef1f4; padding: 1px
3px; border-radius: 3px; font-size: 8.8pt; } pre { background:
#0f1b26; color: #dfe7ee; padding: 8px 10px; border-radius: 5px; font-
size: 8.4pt; overflow-x: auto; white-space: pre-wrap; } pre code {
background: transparent; color: inherit; }
```

CROSS_SIMULATOR_AND_UVM_README

Cross-Simulator & UVM README — O-RAN VIP Suite

Author: AVIK MAJUMDAR · **Project:** AVIK VIP FACTORY — O-RAN VIP Suite

Three execution lanes; one product. All commands via make (see make help) or ./compile.sh.

Lane	What	Simulator(s)	CRV	Coverage
Lane-1	plain-SV VIP (pack/unpack, legal-space, negative)	Verilator 5.020/5.050	Z3 pre-gen	Python func + verilated code
Lane-2A	UVM component architecture (muvvm_pkg)	Verilator 5.020/5.050	Z3 seeded	Python func
Lane-2B	NATIVE UVM (Accellera)	Verilator 5.050 , VCS, Questa, Xcelium	native randomize() with{}	native covergroup (cross+illegal_bin

There is **no Lane-3**. “Lane-2” is the UVM tier, split A (subset, free) / B (native, full).

1. Toolchain — Verilator 5.050 (mandated primary)

```
make verilator050          # git v5.050 source build (~25 min, 2
cores) -> /usr/local/bin/verilator
verilator --version        # Verilator 5.050 2026-07-01 rev v5.050
make uvm-setup             # clone Accellera uvm-core + patch (DPI
shim) into $UVM_HOME
```

UVM-on-Verilator shim (lane2/uvm050/): no-op HDL backdoor stubs matching uvm_dpi.h (char*,p_vpi_vecval), disabled VPI polling include, and vpi_stub.cc providing vpi_get_vlog_info. This is why full Accellera UVM elaborates + runs on Verilator 5.050.

2. Standalone run (single component / test)

```
# Lane-1 (Verilator): one component, one seed
make run COMPONENT=uplane TEST=smoke SEED=1
# Lane-2B native UVM on 5.050: build once, run one test
make lane2b SIM=verilator5050
cd lane2/lane2b_native/suite && ./obj_suite2/simv_uvm
+UVM_TESTNAME=uplane_test +ntb_random_seed=1
# Lane-2B on a licensed sim (source-ready, same UVM):
make lane2b SIM=vcs SEED=1          # or SIM=questa | SIM=xcelium
```

3. Regression

```
make regress-l1 COMPONENT=all          # Lane-1 smoke (seeds
1,2,3) all 13
make regress RAND=100                  # ALL lanes full ->
combined_regression.json
make lane2b SIM=verilator5050          # Lane-2B native UVM
regression (13 tests) on 5.050
```

4. Coverage — CODE and FUNCTIONAL

make coverage

- **Functional** (Lane-1): monitor-log COV-### engine -> regression/suite_regression_full.json (func_cov per component); merged across smoke+random+directed+corner. Native covergroup (Lane-2B) sampled in-sim (cross + illegal bins).
- **Code** (stmt/branch/toggle): custom sim_main.cpp calls coverage()->write("coverage.dat") (GAP-ORAN-003 fix); parsed by gen/code_cov.py -> regression/code_cov.json (stmt 100%, DUT branch 100%, TB-defensive waived).
- Report + dashboard: docs/FULL_COVERAGE_REPORT.md, docs/COVERAGE_DASHBOARD.html.
- Licensed sims: VCS -cm line+cond+fsm+branch+tgl+assert + urg; Questa +cover -coveropt 3
 - vcover report; Xcelium -coverage all + imc.

5. Waveforms (per simulator)

```
make waves SIM=verilator5050 COMPONENT=uplane TEST=smoke SEED=1  #
FST -> gtkwave
make waves SIM=vcs          COMPONENT=uplane          # VPD/FSDB (-
debug_access+all -kdb)
make waves SIM=questa       COMPONENT=uplane          # WLF (-
voptargs+=+acc; add wave -r /*)
make waves SIM=xcelium      COMPONENT=uplane          # SHM (-access +rwc;
probe -create -all)
```

Verilator FST needs --trace-fst (WAVES=1 in run.sh / make waves);
view with gtkwave *.fst.

6. Evidence / result contract

Every run -> result.json via gen/adjudicate.py. STATUS from
executed evidence only (PASS iff sim_exit==0 AND UVM_ERROR==0
AND UVM_FATAL==0 AND transactions>0). Negative:
EXPECTED_FAILURE_DETECTED. Missing log -> NOT_VERIFIED
(never converted).

7. Current portability (executed)

Simulator	Lane-1	Lane-2A	Lane-2B
Verilator 5.020	PASS 1470/1470	PASS 39/39	N/A
Verilator 5.050	PASS	PASS	PASS 13/13 native UVM
VCS / Questa / Xcelium	NOT_RUN	NOT_RUN	source-ready (same UVM)

Combined executed: **1523/1523 = 100% real pass** (incl. RAL uvm_reg register-model test).